

C O D E R + C L A U D E C O D E · Q I N T E S S

Documento TO-BE (Técnico)

Para: Ingeniería / Arquitectura / DevSecOps

De: Equipo DevSecOps — Qintess

Fecha: 2026-07-21

Estado: Diseño aprobado + piloto implementado y validado E2E

Alcance: Piloto squad 15–30 personas (Developer + QA + Architect) →
producción bancaria

Diseño aprobado + piloto E2E

Documento interno · Plataforma de entornos de desarrollo con IA gobernada por rol

0. Cómo leer este documento

Este documento consolida los 14 documentos de diseño en `docs/T0-BE/`. Distingue explícitamente **dos planos**:

- **TO-BE de producción** — el diseño objetivo sobre Kubernetes + Cilium + IdP corporativo.
- **AS-IS del piloto (validado)** — lo que hoy ya funciona en el homelab sobre Docker.

Donde difieren, se marca. La ruta piloto → producción es de **escala e infraestructura, no de rediseño**.

1. Problema

Los operadores del equipo DevSecOps necesitan entornos aumentados con IA (Claude Code) que sean:

- **Gobernados por rol** — cada persona solo ve/hace lo que le corresponde.
- **Aislados en red** — sin acceso a servicios no autorizados (egress restringido).
- **Reproducibles y efímeros** — sin "funciona en mi máquina".
- **IDE web + CLI** — máxima flexibilidad.
- **Auditable** — quién hizo qué, con qué herramientas, cuántos tokens.

2. Solución (arquitectura de referencia)

Plataforma basada en **Coder** que: (1) autentica vía OAuth/OIDC; (2) resuelve el rol automáticamente por pertenencia a equipos/grupos; (3) provisiona un workspace con el **overlay** del rol (CLAUDE.md + skills + MCP + network); (4) expone code-server (VS Code Web) + Claude Code CLI; (5) aplica NetworkPolicies por rol; (6) registra audit trail completo.

```
Operador (browser)
| HTTPS/WSS · Login OAuth/OIDC
▼
Coder Control Plane → Auth (team/group lookup) · Template Engine (Terraform) · Lifecycle
|
▼
Workspace por operador
├─ code-server (VS Code Web, :8080)
├─ Claude Code CLI (terminal / ttyd)
├─ /home/coder/.claude/ ← overlay del rol (CLAUDE.md, mcp-config.json, skills/)
├─ NetworkPolicy: egress FQDN por rol
└─ labels: coder.com/role, coder.com/owner
```

2.1 Enfoque arquitectónico elegido (Decisión #10)

- **Enfoque A — Coder nativo + CLAUDE.md por rol**  (**elegido**). Overlay estático montado por rol.
Pros: mínimo código custom, Coder maneja el lifecycle. *Contras*: overlay estático (rol es build-time).
- **Enfoque B — Sidecar de governance dinámico** . Descartado por complejidad, latencia y superficie de ataque.
- **Enfoque C — Orchestrator centralizado** . Descartado por SPOF y latencia.

3. Registro de decisiones (resumen)

#	Decisión	Elección
1	Cloud provider	Agnostic (implementar en uno)
2	Identity Provider	GitHub OAuth (piloto) → OIDC corporativo (prod)
3	Rol mapping	GitHub Teams / grupos IdP → Coder roles
4	Roles	Architect + Developer + QA (3)
5	LLM	Claude ahora, diseño model-agnostic
6	Interfaz	code-server + Claude Code CLI
7	Template strategy	Híbrido: base + overlay por rol
8	Network isolation	Sí, egress restringido por rol
10	Enfoque arquitectónico	A — Coder nativo + overlay
11	Secretos MCP	Per-user vía OAuth (external-auth) + ESO
12	Jira auth	OAuth 2.0 3LO (1 clic por usuario, rotación auto)
17	Credencial AuthZ del server	GitHub App de la org (installation token auto-rotado)
18	Cambios de rol	Build-time asimétrico: promoción en próximo rebuild; degradación/offboarding inmediato (stop / delete)

4. RBAC y resolución de rol

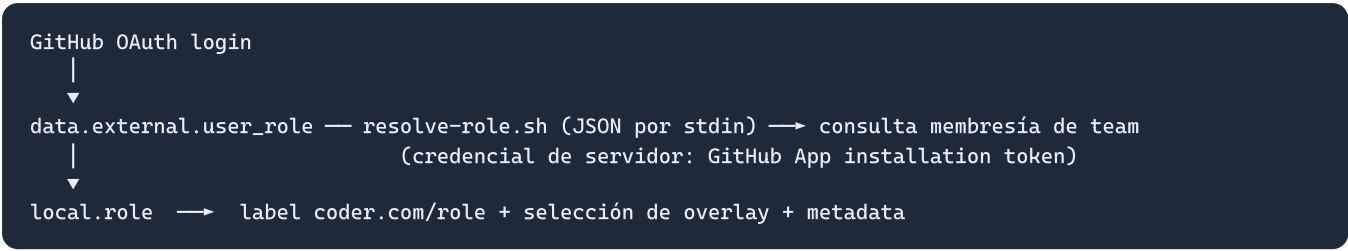
4.1 Mapping de roles

GitHub Team / Grupo IdP	Coder Role	Estado
tech-leads / architects	architect	activo
platform-developers	developer	activo
qa-engineers	qa	activo
devops-sre, cybersecurity, product-managers	devops/security/product	futuro

4.2 Normas de resolución

- **One-role-per-user:** cada persona pertenece a **un solo** team de rol; multi-team es excepción.
- **Least-privilege (red de seguridad):** en multi-team gana el rol **menos permisivo**. Permisividad qa < developer < architect → orden de selección qa → developer → architect.
- **Fail-closed:** sin team válido → unknown (mínimos privilegios), nunca un rol por defecto.

4.3 Mecánica (piloto, validado)



- `resolve-role.sh` lee **JSON por stdin** (`{github_username, github_org}`), consulta la membresía vía **curl + bearer** (en el provisioner de systemd `gh` no está en PATH) y aplica la prioridad least-privilege.
- Precedencia de credencial: **PAT env > GitHub App > gh ambiente** (backward-compat, fail-closed a `unknown`).

4.4 Aplicación de cambios de rol (Decisión #18)

El rol es **build-time**: un contenedor conserva su rol aunque cambie la membresía; solo se re-resuelve al reconstruir (`coder restart / update`). Un reboot con `unless-stopped` **no** re-resuelve.

Cambio	Riesgo	Aplicación
Promoción (qa → architect)	Bajo	Próximo rebuild natural
Degradación / offboarding	Alto	Inmediato, fail-closed: <code>coder stop / delete</code> (no relabel en caliente)

- **Homelab:** mitigación por runbook + `scripts/reconcile-roles.sh` (control loop systemd cada 15 min, `DRY_RUN=1` por defecto; 404 en org → stop+delete; menos-permisivo → restart).
- **Producción:** webhook de la org GitHub (`membership / organization`) → API Coder (reutiliza la GitHub App).

5. Matriz de permisos por rol

Recurso	Architect	Developer	QA
Skills	spec-from-jira, architecture-review	spec-writer, spec-implementer, claude-md-updater, (frontend/UX: framer-motion, ui-ux-pro-max, ui-styling, design-system)	bdd-test-generation, coverage-gap-analysis, unit-test-generation
MCP GitHub	Read/Write	Read/Write	Read-only
MCP Jira	Read	Read + transitions	Read + create bugs
MCP Confluence	Read/Write	—	—
MCP SonarCloud	—	(pendiente cuenta)	—
MCP Playwright	—	—	Execute
Egress	github, anthropic, atlassian	+ npm, pypi, docker, sonarcloud	+ selenium-grid
Governance gates	Spec completo	BDD-first, scope limits (15 files/500 líneas), single-story	Coverage ≥80%, calidad de tests, no PII

6. Template Terraform (estrategia base + overlay)

```
templates/  
├─ mvp-embedded/      # PILOTO (Docker, sin módulos) – activo  
│   ├── main.tf        # docker_container + coder_agent + coder_app + external role + overlay  
│   ├── scripts/       # resolve-role.sh, gh-app-token.sh, gh-app-token-json.sh  
│   └─ ...  
├─ prod-oidc/         # PRODUCCIÓN (K8s, rol nativo por grupo OIDC) – scaffold  
└─ overlays/  
    ├── architect/     { CLAUDE.md, mcp-config.json, skills/ }  
    ├── developer/     { CLAUDE.md, mcp-config.json, skills/ }  
    └─ qa/              { CLAUDE.md, mcp-config.json, skills/ }
```

- **Producción (prod-oidc):** `kubernetes_pod` con overlay vía **ConfigMap** (`overlay-<role>`), montado RO en `/home/coder/.claude` ; `kubernetes_network_policy` de egress por rol; imagen `ghcr.io/qintess/coder-workspace` .
- **Piloto (mvp-embedded):** `docker_container` con `restart="unless-stopped"` (persistencia N6); overlay entregado por **bind-mount** del host (`/opt/overlays` RO) → copiado a `~/.claude/` en el `startup_script` ; sin módulos de registry (evita el bloqueo TF 1.5.7 vs. ≥1.9). El `startup_script` instala code-server (tarball), Claude (npm), Node 20 (NodeSource, requerido por `@playwright/mcp`), `uv/uvx` , chromium (si el rol usa Playwright).

6.1 Overlay → configuración real de Claude Code

- `CLAUDE.md` → `~/.claude/CLAUDE.md`
- `skills/` → `~/.claude/skills/`

- `mcp-config.json` → `~/workspace/.mcp.json` + `~/.claude/settings.json`
(`enableAllProjectMcpServers:true`)
- El agente recibe `CODER_ROLE=local.role` ; ttyd lanza `claude` desde `~/workspace` para tomar el `.mcp.json` .

7. Skills y flujo de trabajo (spec como contrato único)

Un único spec sirve a los 3 roles; cada uno lee sus secciones:

```
PM crea Jira Story
  ▼ ARCHITECT · spec-from-jira      → specs/NN-feature.md
  ▼ QA · bdd-test-generation        → features/<KEY>-<slug>.feature (BDD-first gate)
  ▼ DEVELOPER · spec-implementer    → código + tests + commits (TDD)
  ▼ DEVELOPER · claude-md-updater    → actualiza CLAUDE.md del proyecto
```

Cada skill sigue el patrón `SKILL.md` (frontmatter + rol + guardrails + misión + pasos + pre-flight) + `README.md` + `references/` .

8. MCP y gestión de secretos

Principio: separar credencial de **plataforma** (AuthZ del server) de credencial **por usuario** (acceso a sus recursos). Estado implementado en el piloto:

Server	Paquete	Auth	Estado
github	<code>@modelcontextprotocol/server-github</code>	external-auth OAuth per-usuario (<code>\${GITHUB_TOKEN}</code>) → solo repos del usuario	✅ conecta
atlassian (Jira+Confluence)	<code>uvx mcp-atlassian</code> (OAuth/BYOT)	OAuth 3LO per-usuario (external-auth <code>atlassian</code> , <code>cloud_id</code> autodetectado)	✅ conecta
playwright	<code>@playwright/mcp --headless --isolated</code>	browser local	✅ conecta (Node 20 + chromium)
sonarcloud	placeholder	<code>\${SONAR_TOKEN}</code>	⌚ falta cuenta SonarCloud

- **Tres credenciales GitHub distintas, no confundir:** (1) **OAuth App "coder local"** = login Coder (AuthN); (2) **GitHub App** `coder-rbac-resolver` = resolver rol (AuthZ, permiso único `Members:read` , token auto-rotado ~1h); (3) **external-auth OAuth** = token del usuario para su git/GitHub MCP (solo sus repos).
- **Aislamiento per-usuario demostrado E2E:** mismo template, usuario A ve 90 repos, usuario B ve 0 — el acceso lo aplica GitHub según la identidad del token, no la plataforma.
- **Atlassian per-usuario:** external-auth OAuth (`ATLASSIAN_OAUTH_ENABLE=true` + `ATLASSIAN_OAUTH_CLOUD_ID` + `ATLASSIAN_OAUTH_ACCESS_TOKEN`); patrón reutilizable documentado en `docs/external-auth-oauth-pattern.md` .

- **Producción:** External Secrets Operator (ESO) para credenciales compartidas (Sonar/Selenium); OAuth per-usuario para GitHub/Jira.

9. Backend LLM (model-agnostic, Decisión #5)

Selector `local.llm_env` en el template con precedencia **vertex > bedrock > subscription > api_key**; setea solo las envs del backend elegido:

- `subscription` → `CLAUDE_CODE_OAUTH_TOKEN` (Claude Pro/Max vía `claude setup-token`)
- `vertex` → `CLAUDE_CODE_USE_VERTEX=1` + `ANTHROPIC_VERTEX_PROJECT_ID` + `CLOUD_ML_REGION`
- `bedrock` / `api_key` → envs correspondientes

Config central en `mvp-embedded-vars.yaml` (`llm_backend` , tokens). Multi-user real = Vertex/Bedrock (billing central) o token por usuario. *Pendiente:* token Pro dio 401 → regenerar `setup-token` o usar API key.

10. Network isolation (producción)

- **CNI:** Cilium — egress filtering por **FQDN** (no IPs estáticas).
- **Default deny + inter-pod blocked** (dev ↔ qa aislados) + **ingress solo desde Coder control plane + DNS restringido** a kube-dns.
- Egress permitido por rol (extracto): todos → `api.github.com` , `api.anthropic.com` ; developer → `+npm` , `pypi` , `docker` , `sonarcloud` ; qa → `*.atlassian.net` , `selenium-grid` .
- *Nota piloto:* el aislamiento de red por Cilium es del plano de producción; el homelab (Docker, 1 host) usa el aislamiento por identidad de token + separación de contenedores.

11. Imagen base

- Ubuntu 24.04, multi-stage (~450 MB): code-server, Claude Code CLI, git, gh, Node.js, Python 3, Docker CLI.
- Non-root (`coder` , UID 1000). `HEALTHCHECK` sobre `:8080/healthz` .
- CI/CD: GitHub Actions → `ghcr.io/qintess/coder-workspace:latest` . Rebuild mensual.
- *Gotcha piloto documentado:* el node embebido de code-server puede quedar en 0 bytes → guard endurecido en el `startup_script` (`code-server --version` + `test -s ... /lib/node` + `curl --retry`).

12. Observabilidad y auditoría

Cuatro capas: **Coder Audit** (logins, lifecycle, cambios de template), **Claude usage** (tokens, tool/MCP calls, sesiones), **K8s metrics** (CPU/RAM, egress, policy denies), **Governance events** (gate violations, blocked egress).

- **Stack:** Prometheus + Grafana + Loki; audit trail en PostgreSQL; alertas en Grafana.
- **Retención:** login/workspace 1 año · Claude sessions 90 días · governance violations 1 año · network denies 30 días.
- **KPIs:** `workspace_active_count` , `claude_tokens_total` /user/día (< budget), `governance_gate_violations` → 0 , `network_policy_denied_total` , `workspace_cost_usd` /user/día.

13. AuthZ a escala (homelab → miles de usuarios)

Separación estricta **AuthN** (SSO, escala sola — el usuario nunca pega un token) vs **AuthZ** (rol, resuelto por una credencial de servidor para todos). Ruta de migración:

Nivel	Credencial AuthZ	Atada a	Rotación	Cuándo
0. PAT personal	PAT en <code>server.env</code>	Persona	Manual	Homelab (superado)
1. Bot + fine-grained PAT	PAT <code>read:org</code> de cuenta bot	Bot	Anual	Piloto/squad
2. GitHub App ✓	Installation token (JWT → ~1h)	La App en la org	Auto ~1h	Producción con GitHub (elegido)
3. IdP + group sync (OIDC/SCIM)	Ninguna custom (claims del IdP)	IdP corporativo	N/A	Banco/miles (futuro)

- **Nivel 2 (elegido, Camino A):** `resolve-role.sh` se mantiene; la credencial pasa a **GitHub App** (`gh-app-token.sh` : JWT RS256 → installation token). No atada a persona, auditable, rate limits por instalación.
- **Nivel 3 (scaffold `prod-oidc`):** elimina `resolve-role.sh` ; rol nativo por `data.coder.workspace_owner.me.groups` → template ACL; SCIM para alta/baja automática. Requiere IdP OIDC real + Coder Premium (group sync avanzado).

14. Deployment y operación (producción)

Pre-requisitos: K8s v1.28+ con Cilium · Helm v3 · Terraform v1.5+ · GitHub Org con Teams (u OIDC) · DNS+TLS (`coder.banco-internal.com` , cert-manager) · PVC para workspaces · External Secrets Operator.

```
helm install coder coder-v2/coder -n coder --values values.yaml
coder templates push workspace-rbac --directory ./coder-templates/ --yes
kubectl create configmap overlay-developer --from-file=overlays/developer/ -n coder-workspaces
# ... overlay-qa, overlay-architect
```

Day-2: imagen mensual · rotar secrets 90 días · autoscaler · backup PostgreSQL diario · audit review semanal · auto-stop 2h idle.

Persistencia (piloto, N6 implementado): server como `systemd --user` (`coder.service` + linger) y contenedores `restart="unless-stopped"` → sobreviven a reboot del host (tradeoff: contenedor de larga vida, apropiado homelab).

15. Rollout (8 semanas)

Semana	Actividad
1	Deploy Coder + imagen base
2	GitHub OAuth + role resolution
3	Overlays + NetworkPolicies
4	5 early adopters (3 dev + 2 QA)
5–6	Feedback y ajustes
7	Rollout squad completo (15–30)
8	Review métricas y KPIs

16. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
IdP/OAuth caído	No login	Admin local de emergencia
Quota/costo Claude	IA degradada	Budget alerts + graceful degradation + auto-stop
Overlay desactualizado	Permisos incorrectos	Rebuild de ConfigMaps en merge a main (CI/CD)
Misconfiguración Cilium	Egress leak	Integration tests + auditoría periódica de políticas
Cost overrun	Presupuesto	Auto-stop + alertas de compute en Grafana
Rol residual tras baja	Hallazgo de auditoría	Decisión #18: offboarding = stop/delete inmediato + reconciliador/webhook

17. Decisiones de no-hacer (YAGNI)

❌ Sidecar de governance dinámico · ❌ Orchestrator centralizado (SPOF) · ❌ Multi-cloud simultáneo · ❌ >3 roles en el piloto · ❌ Extensiones de IDE custom.

18. Estado del piloto (validado E2E, homelab)

Capacidad	Estado
Coder v2.35.1 standalone + VS Code + Claude Code	✓
Login GitHub OAuth	✓
RBAC 3 roles (developer/qa/architect) con cuentas reales	✓ (JuanParralV, susanacarolina)
GitHub App para AuthZ (rol)	✓ activada y validada
MCP GitHub / Atlassian / Playwright per-usuario	✓ conectados
Aislamiento per-usuario (90 vs 0 repos)	✓ demostrado
Overlays por rol (skills+MCP+persona)	✓
Persistencia N6 (systemd + unless-stopped)	✓ verificado tras reboot
Config central para cambiar de org/cliente	✓
prod-oidc (K8s)	🟡 scaffold, no desplegado
Backend LLM autenticado	⌚ token Pro 401 (regenerar) o API key
SonarCloud MCP	⌚ sin cuenta

Anexo — Documentos fuente en [docs/T0-BE/](#)

[00-desiciones](#) · [01-arquitectura-general](#) · [02-rbac-mapping-roles](#) · [03-template-terraform](#) · [04-skills-mcp-por-rol](#) · [05-network-isolation](#) · [06-imagen-base-docker](#) · [07-observabilidad-audit](#) · [08-deployment-operacion](#) · [09-gestion-secretos](#) · [10-jira-oauth-automatico](#) · [11-analisis-para-crear-specs](#) · [12-roles-y-skilles](#) · [13-authz-a-escala](#) · [SPEC-coder-claude-rbac-platform](#) . Runbooks: [RUNBOOK](#) , [RUNBOOK-github-app](#) , [RUNBOOK-prod-oidc](#) , [RUNBOOK-llm-backend](#) , [CONFIGURACION](#) , [external-auth-oauth-pattern](#) .